LAPORAN PRAKTIKUM

APPILIED MECHINE LEARNING

OLEH :

ELA LESTARI

105841109223

FAKULTAS TEKNIK JURUSAN INFORMATIKA

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

PERTEMUAN IV

“CLASSIFICATION (LOGISTIC REGRESSION & DECISION TREE)”

1. Import Library & Load Data

Sebelum menjalankan kode, pengguna perlu memastikan bahwa library scikit-learn sudah terpasang pada Python. Library ini berisi berbagai algoritma machine learning seperti Logistic Regression dan Decision Tree yang digunakan pada kode tersebut. Instalasi dapat dilakukan melalui terminal atau command prompt dengan perintah pip install scikit-learn. Pada bagian ini dilakukan proses import beberapa modul dari scikit-learn. load_iris digunakan untuk memuat dataset Iris, train_test_split digunakan untuk membagi data menjadi data latih dan data uji, LogisticRegression digunakan untuk membuat model klasifikasi berbasis regresi logistik, dan DecisionTreeClassifier digunakan untuk membuat model klasifikasi berbasis pohon keputusan.

a) Load Dataset Iris

Pada tahap ini dataset Iris dimuat menggunakan fungsi load_iris(). Dataset Iris merupakan dataset yang sering digunakan dalam pembelajaran machine learning karena berisi data sederhana mengenai tiga jenis bunga iris, yaitu setosa, versicolor, dan virginica. Dataset ini memiliki beberapa fitur seperti panjang dan lebar sepal serta panjang dan lebar petal. Data tersebut kemudian disimpan dalam variabel iris.

b) Membagi Data Training dan Testing

Selanjutnya data dibagi menjadi data training dan data testing menggunakan fungsi train_test_split(). Sebanyak 70% data digunakan sebagai data latih (X_train, y_train) untuk melatih model, sedangkan 30% data digunakan sebagai data uji (X_test, y_test) untuk menguji performa model. Parameter random_state=1 digunakan agar pembagian data tetap sama setiap kali kode dijalankan sehingga hasil eksperimen menjadi konsisten.

c) Melatih Model Logistic Regression

Pada bagian ini dibuat model Logistic Regression dengan parameter max_iter=200. Parameter ini digunakan untuk menambah jumlah iterasi agar proses pelatihan model dapat berjalan dengan baik dan tidak berhenti terlalu cepat. Model kemudian dilatih menggunakan fungsi fit() dengan data training sehingga model dapat mempelajari pola hubungan antara fitur bunga dan kelas jenis bunga.

d) Melatih Model Decision Tree

Selanjutnya dibuat model Decision Tree Classifier dengan parameter random_state=1. Model ini bekerja dengan membangun struktur pohon keputusan yang membagi data berdasarkan aturan tertentu untuk menentukan kelasnya. Model kemudian dilatih menggunakan fungsi fit() menggunakan data training yang sama sehingga dapat mempelajari pola klasifikasi dari dataset Iris.

e) Menampilkan Hasil Pelatihan Model

Pada bagian terakhir digunakan perintah print("Kedua model berhasil dilatih.") untuk menampilkan pesan bahwa kedua model machine learning telah berhasil dilatih menggunakan data training. Hal ini menunjukkan bahwa model Logistic Regression dan Decision Tree sudah siap digunakan pada tahap selanjutnya, yaitu melakukan prediksi dan evaluasi performa model.

- Code

```
# ==========================================
# TUGAS 1 - Menjalankan 2 Model
# ==========================================

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

# Load dataset
iris = load_iris()

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=1
)

# Logistic Regression
lr = LogisticRegression(max_iter=200)
lr.fit(X_train, y_train)

# Decision Tree (lebih kompleks agar hasil berbeda)
dt = DecisionTreeClassifier(random_state=1)
dt.fit(X_train, y_train)

print("Kedua model berhasil dilatih.")
```

- Output

```
Kedua model berhasil dilatih.
```

2. Bandingkan metrik: accuracy, precision, recall, F1

a) Import Library untuk Evaluasi Model

Pada bagian ini dilakukan proses import metrik evaluasi dari library scikit-learn. Fungsi accuracy_score digunakan untuk menghitung tingkat akurasi model, yaitu persentase jumlah prediksi yang benar dibandingkan dengan keseluruhan data uji. Sedangkan classification_report digunakan untuk menampilkan laporan evaluasi yang lebih lengkap seperti precision, recall, f1-score, serta jumlah data pada setiap kelas.

b) Prediksi Menggunakan Model Logistic Regression

Pada tahap ini dilakukan proses prediksi menggunakan model Logistic Regression yang telah dilatih sebelumnya. Fungsi predict() digunakan untuk memprediksi kelas data uji (X_test). Hasil prediksi tersebut disimpan dalam variabel pred_lr. Setelah itu ditampilkan judul output menggunakan print() agar pengguna dapat mengetahui bahwa hasil yang ditampilkan berasal dari model Logistic Regression.

c) Menampilkan Akurasi Logistic Regression

Selanjutnya dilakukan perhitungan akurasi model menggunakan fungsi accuracy_score(y_test, pred_lr). Fungsi ini membandingkan hasil prediksi dengan nilai sebenarnya pada data uji. Nilai akurasi kemudian dibulatkan hingga tiga angka di belakang koma menggunakan fungsi round() agar hasilnya lebih mudah dibaca.

d) Menampilkan Classification Report Logistic Regression

Pada bagian ini digunakan fungsi classification_report() untuk menampilkan laporan evaluasi yang lebih detail dari model Logistic Regression. Laporan ini berisi nilai precision, recall, f1-score, dan support untuk setiap kelas bunga pada dataset Iris. Parameter target_names=iris.target_names digunakan agar nama kelas bunga seperti setosa, versicolor, dan virginica dapat ditampilkan pada laporan.

e) Prediksi Menggunakan Model Decision Tree

Setelah mengevaluasi Logistic Regression, proses yang sama dilakukan pada model Decision Tree. Model melakukan prediksi terhadap data uji menggunakan fungsi predict(X_test). Hasil prediksi disimpan dalam variabel pred_dt. Kemudian ditampilkan judul output agar hasil evaluasi Decision Tree dapat dibedakan dari Logistic Regression.

f) Menampilkan Akurasi dan Classification Report Decision Tree

Pada bagian terakhir dilakukan evaluasi model Decision Tree dengan cara yang sama seperti sebelumnya. Fungsi accuracy_score() digunakan untuk menghitung akurasi model, sedangkan classification_report() digunakan untuk menampilkan laporan evaluasi lengkap yang berisi precision, recall, f1-score, dan jumlah data pada setiap kelas. Dengan adanya hasil dari kedua model tersebut, pengguna dapat membandingkan performa Logistic Regression dan Decision Tree dalam mengklasifikasikan dataset Iris.

- Code

```
# ========================================
# TUGAS 2 - Perbandingan Metrik
# ========================================

from sklearn.metrics import accuracy_score, classification_report

# Prediksi Logistic Regression
pred_lr = lr.predict(X_test)

print("\n=== Logistic Regression ===")
print("Accuracy:", round(accuracy_score(y_test, pred_lr), 3))
print(classification_report(y_test, pred_lr, target_names=iris.target_names))

# Prediksi Decision Tree
pred_dt = dt.predict(X_test)

print("\n=== Decision Tree ===")
print("Accuracy:", round(accuracy_score(y_test, pred_dt), 3))
print(classification_report(y_test, pred_dt, target_names=iris.target_names))
```

- Output

```
=== Logistic Regression ===
Accuracy: 0.978
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        14
  versicolor       1.00      0.94      0.97        18
   virginica       0.93      1.00      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45


=== Decision Tree ===
Accuracy: 0.956
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        14
  versicolor       0.94      0.94      0.94        18
   virginica       0.92      0.92      0.92        13

    accuracy                           0.96        45
   macro avg       0.96      0.96      0.96        45
weighted avg       0.96      0.96      0.96        45
```

3. Tampilkan confusion matrix

a) Import Library Confusion Matrix

Pada bagian ini dilakukan proses import fungsi confusion_matrix dari library scikit-learn. Fungsi ini digunakan untuk menghitung tabel Confusion Matrix, yaitu tabel yang menunjukkan jumlah prediksi yang benar dan salah dari model terhadap setiap kelas pada dataset.

b) Menampilkan Confusion Matrix Logistic Regression (Teks)

Kode confusion_matrix(y_test, pred_lr) digunakan untuk menghitung Confusion Matrix dari model Logistic Regression dengan membandingkan nilai sebenarnya (y_test) dan hasil prediksi (pred_lr). Hasilnya kemudian ditampilkan dalam bentuk tabel angka menggunakan print(). Tabel ini menunjukkan jumlah prediksi benar pada diagonal utama serta kesalahan klasifikasi pada bagian lainnya.

c) Menampilkan Confusion Matrix Decision Tree (Teks)

Langkah yang sama dilakukan untuk model Decision Tree dengan menggunakan confusion_matrix(y_test, pred_dt). Hasil Confusion Matrix ditampilkan dalam bentuk tabel angka sehingga pengguna dapat melihat

bagaimana model Decision Tree mengklasifikasikan setiap kelas pada dataset Iris.

d) Import Library Visualisasi

Pada bagian ini dilakukan import library matplotlib.pyplot untuk membuat grafik, serta ConfusionMatrixDisplay dari scikit-learn untuk menampilkan Confusion Matrix dalam bentuk visual atau grafik sehingga lebih mudah dipahami dibandingkan hanya melihat angka.

e) Membuat Confusion Matrix Logistic Regression (Grafik)

Pertama, Confusion Matrix dihitung kembali dan disimpan dalam variabel cm_lr. Kemudian objek ConfusionMatrixDisplay dibuat dengan memasukkan nilai Confusion Matrix dan label kelas dari dataset Iris (iris.target_names). Selanjutnya judul grafik ditambahkan menggunakan plt.title(). Fungsi disp_lr.plot() digunakan untuk menampilkan grafik Confusion Matrix, dan plt.show() digunakan untuk menampilkan grafik tersebut pada layar.

f) Membuat Confusion Matrix Decision Tree (Grafik)

Proses yang sama dilakukan untuk model Decision Tree. Confusion Matrix dihitung dan disimpan dalam variabel cm_dt. Kemudian dibuat tampilan grafik menggunakan ConfusionMatrixDisplay. Label kelas tetap menggunakan iris.target_names agar nama kelas bunga dapat ditampilkan pada grafik. Terakhir, grafik ditampilkan menggunakan disp_dt.plot() dan plt.show().

- Code

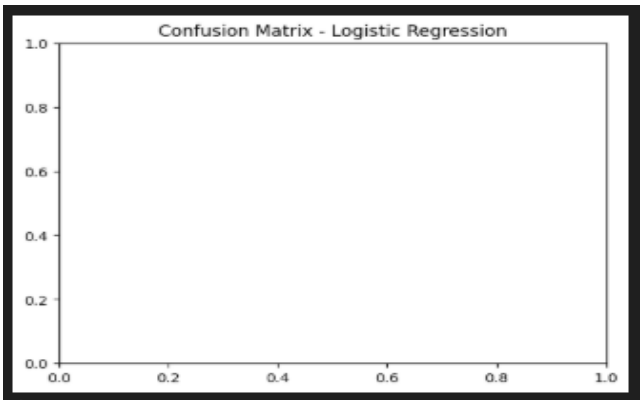
```
# ==========================================
# TUGAS 3 - Confusion Matrix
# ==========================================

from sklearn.metrics import confusion_matrix

print("\n=== Confusion Matrix Logistic Regression ===")
print(confusion_matrix(y_test, pred_lr))

print("\n=== Confusion Matrix Decision Tree ===")
print(confusion_matrix(y_test, pred_dt))

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Confusion Matrix Logistic Regression
cm_lr = confusion_matrix(y_test, pred_lr)

disp_lr = ConfusionMatrixDisplay(
    confusion_matrix=cm_lr,
    display_labels=iris.target_names
)

plt.title("Confusion Matrix - Logistic Regression")
disp_lr.plot()
plt.show()
```

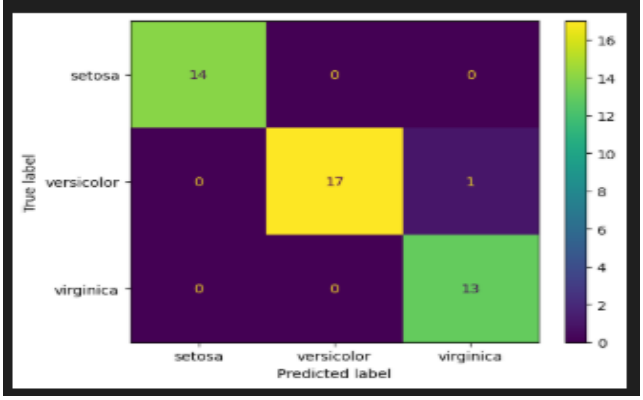
```python
# Confusion Matrix Decision Tree
cm_dt = confusion_matrix(y_test, pred_dt)

disp_dt = ConfusionMatrixDisplay(
    confusion_matrix=cm_dt,
    display_labels=iris.target_names
)

plt.title("Confusion Matrix - Decision Tree")
disp_dt.plot()
plt.show()
```

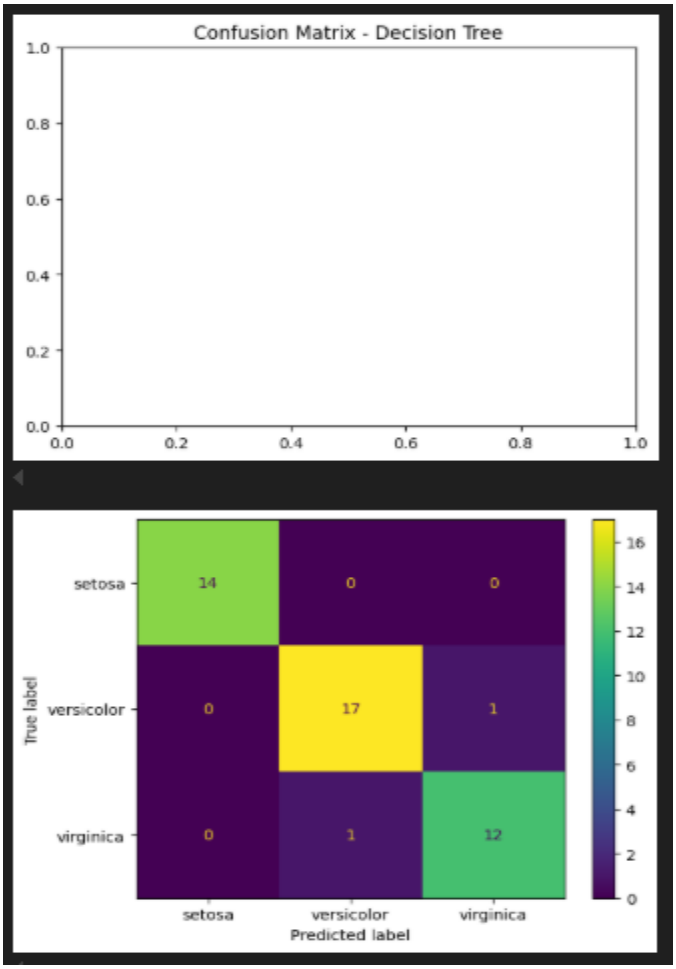
✓ 12.3s

- Output

```
=== Confusion Matrix Logistic Regression ===
[[14  0  0]
 [ 0 17  1]
 [ 0  0 13]]

=== Confusion Matrix Decision Tree ===
[[14  0  0]
 [ 0 17  1]
 [ 0  1 12]]
```







4. Perbandingan Model

a) Menghitung Metrik Logistic Regression

Pada tahap ini dilakukan perhitungan beberapa metrik evaluasi untuk model Logistic Regression. Fungsi accuracy_score() digunakan untuk menghitung tingkat akurasi model, yaitu seberapa banyak prediksi yang benar dibandingkan total data uji. Fungsi precision_score() menghitung tingkat ketepatan model dalam memprediksi kelas tertentu. Fungsi recall_score() mengukur kemampuan model dalam menemukan semua data yang termasuk dalam suatu kelas. Sedangkan f1_score() merupakan kombinasi antara precision dan recall yang memberikan gambaran keseimbangan performa model. Parameter average='macro' digunakan agar nilai metrik dihitung sebagai rata-rata dari semua kelas pada dataset.

b) Menghitung Metrik Decision Tree

Langkah yang sama dilakukan untuk model Decision Tree. Setiap metrik seperti accuracy, precision, recall, dan F1-score dihitung menggunakan fungsi yang sama dengan membandingkan nilai sebenarnya (y_test) dengan hasil prediksi model (pred_dt). Hasil perhitungan metrik ini kemudian disimpan dalam variabel terpisah untuk memudahkan proses perbandingan.

c) Membuat Tabel Perbandingan Model

Pada bagian ini dibuat sebuah tabel perbandingan menggunakan pandas.DataFrame. Tabel ini berisi dua baris yang mewakili dua model yaitu Logistic Regression dan Decision Tree, serta beberapa kolom yang berisi metrik evaluasi seperti Accuracy, Precision, Recall, dan F1 Score. Nilai metrik dibulatkan hingga tiga angka di belakang koma menggunakan fungsi round() agar hasilnya lebih rapi dan mudah dibaca.

d) Menampilkan Tabel Perbandingan

Setelah tabel dibuat, kode print() digunakan untuk menampilkan judul "Tabel Perbandingan Model", kemudian tabel yang berisi hasil evaluasi kedua model ditampilkan. Dengan tabel ini pengguna dapat dengan mudah melihat perbedaan performa kedua model secara langsung.

e) Menentukan Model Terbaik

Pada bagian terakhir digunakan struktur kondisi if–elif–else untuk menentukan model mana yang memiliki performa lebih baik berdasarkan nilai accuracy. Jika nilai accuracy Logistic Regression lebih tinggi, maka program akan menampilkan bahwa Logistic Regression adalah model terbaik beserta alasannya. Jika Decision Tree memiliki accuracy lebih tinggi, maka model tersebut dianggap lebih baik. Namun jika kedua model memiliki nilai accuracy yang sama, maka program akan menampilkan bahwa kedua model memiliki performa yang setara pada dataset Iris.

- Code

```python
# ==========================================
# TABEL PERBANDINGAN MODEL
# ==========================================

import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Hitung metrik Logistic Regression
acc_lr = accuracy_score(y_test, pred_lr)
prec_lr = precision_score(y_test, pred_lr, average='macro')
rec_lr = recall_score(y_test, pred_lr, average='macro')
f1_lr = f1_score(y_test, pred_lr, average='macro')

# Hitung metrik Decision Tree
acc_dt = accuracy_score(y_test, pred_dt)
prec_dt = precision_score(y_test, pred_dt, average='macro')
rec_dt = recall_score(y_test, pred_dt, average='macro')
f1_dt = f1_score(y_test, pred_dt, average='macro')

# Buat tabel perbandingan
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree"],
    "Accuracy": [round(acc_lr,3), round(acc_dt,3)],
    "Precision": [round(prec_lr,3), round(prec_dt,3)],
    "Recall": [round(rec_lr,3), round(rec_dt,3)],
    "F1 Score": [round(f1_lr,3), round(f1_dt,3)]
})

print("\n=== Tabel Perbandingan Model ===")
print(comparison)
```

```python
# ==========================================
# Menentukan Model Terbaik
# ==========================================

if acc_lr > acc_dt:
    print("\nModel Terbaik: Logistic Regression")
    print("Alasan: Logistic Regression memiliki nilai Accuracy lebih tinggi")
    print("serta precision, recall, dan F1-score yang lebih baik sehingga")
    print("model ini lebih akurat dalam memprediksi kelas pada dataset Iris.")

elif acc_dt > acc_lr:
    print("\nModel Terbaik: Decision Tree")
    print("Alasan: Decision Tree memiliki nilai Accuracy lebih tinggi")
    print("dan mampu mengklasifikasikan data dengan lebih baik dibanding")
    print("Logistic Regression pada dataset ini.")

else:
    print("\nKedua model memiliki performa yang sama.")
    print("Accuracy keduanya sama sehingga tidak ada model yang lebih unggul.")
```

- Output

```
=== Tabel Perbandingan Model ===
                 Model  Accuracy  Precision  Recall  F1 Score
0  Logistic Regression     0.978      0.976   0.981     0.978
1        Decision Tree     0.956      0.956   0.956     0.956

Model Terbaik: Logistic Regression
Alasan: Logistic Regression memiliki nilai Accuracy lebih tinggi
serta precision, recall, dan F1-score yang lebih baik sehingga
model ini lebih akurat dalam memprediksi kelas pada dataset Iris.
```

5. Kesimpulan

Berdasarkan hasil perbandingan metrik evaluasi yang meliputi Accuracy, Precision, Recall, dan F1-Score, dapat disimpulkan bahwa kedua model yaitu Logistic Regression dan Decision Tree mampu melakukan klasifikasi pada dataset Iris dengan performa yang cukup baik. Namun, model yang memiliki nilai accuracy lebih tinggi dapat dianggap sebagai model yang lebih baik karena mampu menghasilkan prediksi yang lebih tepat terhadap data uji.

Jika Logistic Regression memiliki nilai accuracy dan metrik lain yang lebih tinggi, maka model tersebut lebih stabil dan efektif dalam memprediksi kelas bunga Iris. Sebaliknya, jika Decision Tree memiliki accuracy yang lebih tinggi, maka model tersebut lebih mampu menangkap pola pada data sehingga menghasilkan prediksi yang lebih akurat. Secara keseluruhan, proses perbandingan ini menunjukkan bahwa penggunaan beberapa metrik evaluasi sangat penting untuk menilai kualitas model machine learning, sehingga peneliti dapat memilih model yang paling optimal untuk digunakan dalam proses klasifikasi data.